EXPERIMENT 3

Compare Strassen's Method of Matrix Multiplication with the Traditional Method

Name	HIMANSHU RAJ
Batch	B1 CCVT
SAP ID	590026147
Submitted To	Arjun Abhishek

B.Tech CSE – Cloud Computing & Virtualization (CCVT)

1. Aim

To implement Strassen’s matrix multiplication algorithm and the traditional matrix multiplication algorithm in C, and compare their performance in terms of execution time and time complexity.

2. Objectives

• Implement traditional matrix multiplication using three nested loops.

• Implement Strassen’s matrix multiplication using divide and conquer.

• Multiply the same input matrices using both algorithms.

• Verify that both methods produce identical result matrices.

• Measure and compare the execution times and theoretical complexities.

3. Theory

Traditional matrix multiplication calculates each result element by multiplying a row of the first matrix by a column of the second matrix and adding the products. For two n × n matrices, it uses three nested loops and has $O(n^3)$ time complexity.

Strassen’s algorithm divides each matrix into four submatrices and uses seven recursive matrix multiplications rather than eight. This reduces its theoretical time complexity to $O(n^{\log_2 7})$, approximately $O(n^{2.807})$. It requires additional temporary storage and can have greater overhead for small matrices.

4. Algorithm

A. Traditional Matrix Multiplication

1. Read two square matrices A and B of order n.

2. For every row i and column j, initialize C[i][j] to zero.

3. Multiply A[i][k] by B[k][j] for every k and add each product to C[i][j].

4. Display the resulting matrix C.

B. Strassen’s Matrix Multiplication

1. If the matrix order is 1, multiply the two single elements.

2. Divide each matrix into four equal submatrices.

3. Compute seven recursive products using additions and subtractions of submatrices.

4. Combine the seven products to obtain the four submatrices of the result.

5. Repeat recursively until the base case is reached, then combine and display the result.

5. Sample Input and Output

The recorded test used matrix order n = 8.

Matrix A: rows containing the integers 1 through 64 in sequence, arranged in eight rows of eight elements.

Matrix B: an 8 × 8 identity matrix, with 1s on the main diagonal and 0s elsewhere.

Multiplying any matrix by the identity matrix returns the original matrix. Both algorithms displayed the same 8 × 8 result, matching Matrix A, and the program reported: Results match: Yes.

The measured times in the recorded run were 0.00000000 seconds for both methods. This is expected for such a small input with the timer resolution used; it does not mean the computation takes literally zero time.

6. Complexity Analysis

Parameter	Traditional Method	Strassen's Method
Best-case time	$O(n^3)$	O(n^2.807) approx.
Average-case time	$O(n^3)$	O(n^2.807) approx.
Worst-case time	$O(n^3)$	O(n^2.807) approx.
Extra space	O(1) auxiliary*	Higher temporary storage

*Traditional auxiliary space excludes the input and output matrices. The implementation of Strassen's method uses temporary submatrices and recursion, so its extra memory use is higher.

7. Performance Comparison

Both algorithms returned identical results for the tested matrices. The traditional method is simpler and usually efficient for small matrices. Strassen's method has a better asymptotic time complexity for sufficiently large matrices, but temporary arrays and recursive calls can make it slower for small inputs. The measured times for the 8 × 8 test are too small to draw a reliable performance conclusion.

8. Result

The traditional and Strassen matrix multiplication algorithms were implemented in C and tested on the same 8 × 8 matrices. Both generated identical result matrices, and the program displayed "Results match: Yes". Their theoretical time complexities were compared, showing $O(n^3)$ for the traditional method and approximately O(n^2.807) for Strassen's method.

9. Screenshots / Evidence

The following screenshots document the implementation and successful execution in Visual Studio Code. The source-code section is omitted, following the format used for the previous experiment reports.

Screenshot 1

exp3.c > strassen(int, int [n][n], int [n][n], int [n][n])

```c

#include <stdio.h>
#include <time.h>

#define MAX 64

// Add two matrices
Windsurf: Refactor | Explain | ×
void add(int n, int A[n][n], int B[n][n],
         int R[n][n]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            R[i][j] = A[i][j] + B[i][j];
}

// Subtract two matrices
Windsurf: Refactor | Explain | ×
void subtract(int n, int A[n][n], int B[n][n],
              int R[n][n]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            R[i][j] = A[i][j] - B[i][j];
}

// Traditional matrix multiplication
Windsurf: Refactor | Explain | ×
void traditional(int n, int A[n][n], int B[n][n],
                 int C[n][n]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            C[i][j] = 0;
            for (int k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
    }
}

// Strassen's matrix multiplication
Windsurf: Refactor | Explain | ×
void strassen(int n, int A[n][n], int B[n][n],
              int C[n][n]) {
    if (n == 1) {
        C[0][0] = A[0][0] * B[0][0];
        return;
    }

    int k = n / 2;

```

Screenshot 1: Matrix addition, subtraction, traditional multiplication, and the beginning of Strassen's algorithm.

Screenshot 2

```
45      int A11[k][k], A12[k][k], A21[k][k], A22[k][k];
46      int B11[k][k], B12[k][k], B21[k][k], B22[k][k];
47
48      int M1[k][k], M2[k][k], M3[k][k], M4[k][k];
49      int M5[k][k], M6[k][k], M7[k][k];
50      int S[k][k], T[k][k];
51
52      int C11[k][k], C12[k][k], C21[k][k], C22[k][k];
53
54      // Divide A and B into four submatrices
55      for (int i = 0; i < k; i++) {
56          for (int j = 0; j < k; j++) {
57              A11[i][j] = A[i][j];
58              A12[i][j] = A[i][j + k];
59              A21[i][j] = A[i + k][j];
60              A22[i][j] = A[i + k][j + k];
61
62              B11[i][j] = B[i][j];
63              B12[i][j] = B[i][j + k];
64              B21[i][j] = B[i + k][j];
65              B22[i][j] = B[i + k][j + k];
66          }
67      }
68
69      // M1 = (A11 + A22)(B11 + B22)
70      add(k, A11, A22, S);
71      add(k, B11, B22, T);
72      strassen(k, S, T, M1);
73
74      // M2 = (A21 + A22)B11
75      add(k, A21, A22, S);
76      strassen(k, S, B11, M2);
77
78      // M3 = A11(B12 - B22)
79      subtract(k, B12, B22, T);
80      strassen(k, A11, T, M3);
81
82      // M4 = A22(B21 - B11)
83      subtract(k, B21, B11, T);
84      strassen(k, A22, T, M4);
85
86      // M5 = (A11 + A12)B22
87      add(k, A11, A12, S);
88      strassen(k, S, B22, M5);
89
90      // M6 = (A21 - A11)(B11 + B12)
```

Screenshot 2: Division into submatrices and computation of Strassen's seven products.

Screenshot 3

```
    subtract(k, A21, A11, S);
    add(k, B11, B12, T);
    strassen(k, S, T, M6);

    // M7 = (A12 - A22)(B21 + B22)
    subtract(k, A12, A22, S);
    add(k, B21, B22, T);
    strassen(k, S, T, M7);

    // Calculate the four result submatrices
    add(k, M1, M4, S);
    subtract(k, S, M5, T);
    add(k, T, M7, C11);

    add(k, M3, M5, C12);
    add(k, M2, M4, C21);

    subtract(k, M1, M2, S);
    add(k, S, M3, T);
    add(k, T, M6, C22);

    // Combine the four submatrices
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < k; j++) {
            C[i][j] = C11[i][j];
            C[i][j + k] = C12[i][j];
            C[i + k][j] = C21[i][j];
            C[i + k][j + k] = C22[i][j];
        }
    }
}

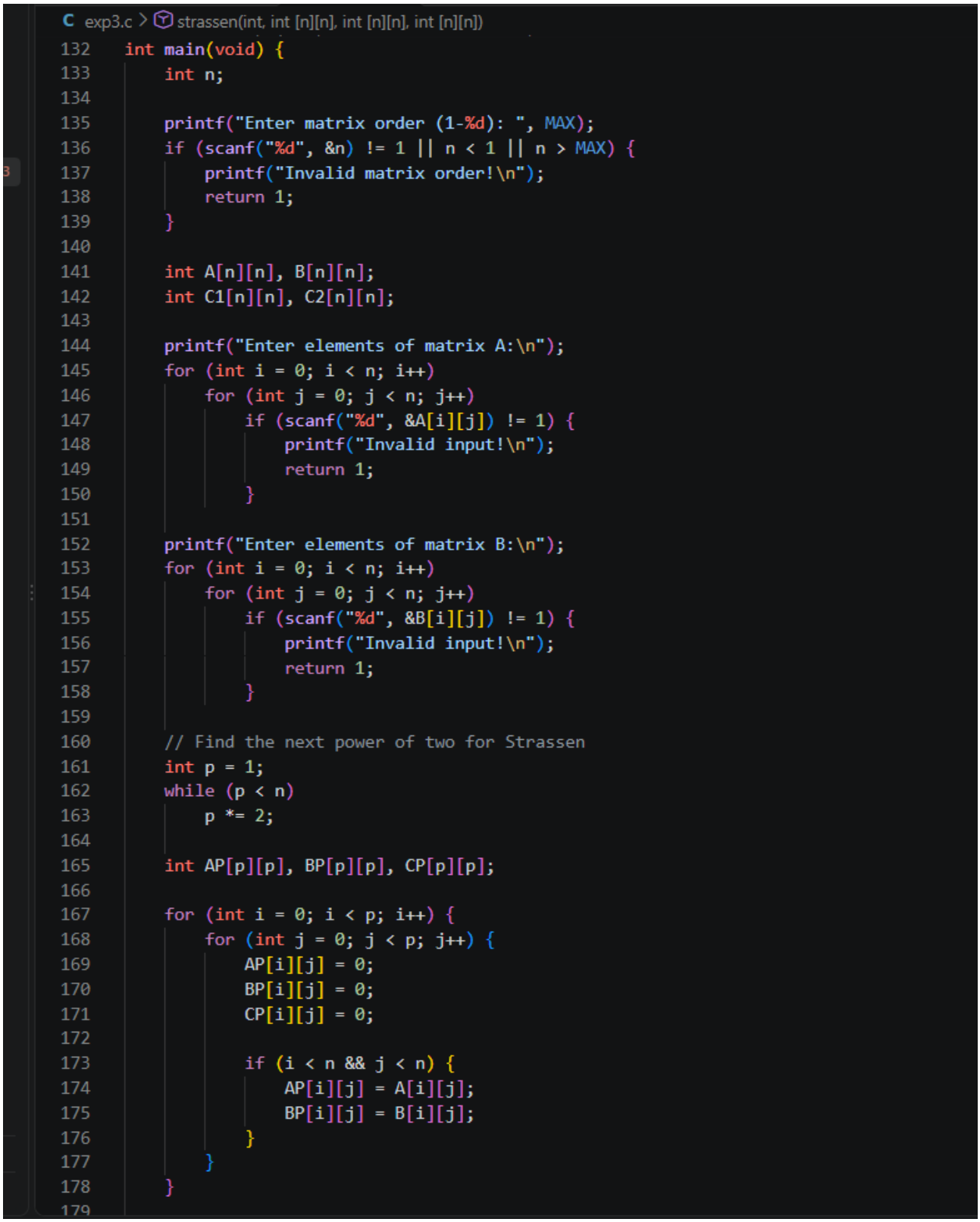
// Display a matrix
Windsurf: Refactor | Explain | ×
void display(int n, int A[n][n]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", A[i][j]);
        printf("\n");
    }
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
int main(void) {
    int n;

    printf("Enter matrix order (1-%d): ", MAX);
```

Screenshot 3: Combination of the seven products and matrix display function.

Screenshot 4



Screenshot 4: Input validation, reading matrices, and padding dimensions for Strassen's method.

Screenshot 5

```
    clock_t start, end;
    double traditionalTime, strassenTime;

    start = clock();
    traditional(n, A, B, C1);
    end = clock();
    traditionalTime = (double)(end - start) / CLOCKS_PER_SEC;

    start = clock();
    strassen(p, AP, BP, CP);
    end = clock();
    strassenTime = (double)(end - start) / CLOCKS_PER_SEC;

    // Copy the original n x n result from padded output
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C2[i][j] = CP[i][j];

    printf("\nTraditional Multiplication Result:\n");
    display(n, C1);

    printf("\nStrassen Multiplication Result:\n");
    display(n, C2);

    int same = 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (C1[i][j] != C2[i][j])
                same = 0;

    printf("\nResults match: %s\n", same ? "Yes" : "No");

    printf("\nPerformance Comparison\n");
    printf("Traditional time: %.8f seconds\n", traditionalTime);
    printf("Strassen time:    %.8f seconds\n", strassenTime);

    return 0;
}
```

Screenshot 5: Timing, output display, result verification, and performance comparison.

Screenshot 6

```
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa> gcc exp3.c -o exp3.exe
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa> .\exp3.exe
Enter matrix order (1-64): 8
Enter elements of matrix A:
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16
17 18 19 20 21 22 23 24
25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48
49 50 51 52 53 54 55 56
57 58 59 60 61 62 63 64
Enter elements of matrix B:
1 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0
0 0 1 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1

Traditional Multiplication Result:
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16
17 18 19 20 21 22 23 24
25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48
49 50 51 52 53 54 55 56
57 58 59 60 61 62 63 64

Strassen Multiplication Result:
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16
17 18 19 20 21 22 23 24
25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48
49 50 51 52 53 54 55 56
57 58 59 60 61 62 63 64

Results match: Yes

Performance Comparison
Traditional time: 0.00000000 seconds
Strassen time:    0.00000000 seconds
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa>
```

Screenshot 6: Successful execution for order 8, showing identical results and the measured execution times.